

AWS cost Report to Slack notification

AWS Cost Explorer • Python • Slack • Automation

THE PROBLEM

I wanted an easier way to see AWS costs without opening the AWS console and checking the report manually every day.

WHAT I WANTED

- See AWS costs by service. Send the report directly to Slack
- Make the information easy to read. Reduce repetitive manual work

HOW I DEBUGGED IT

I started by generating the AWS cost report and checking the output. Then I tested the Slack setup, permissions, and channel configuration step by step until the report was reaching Slack correctly.

THE SOLUTION

I connected AWS Cost Explorer → AWS Cost CLI → Python → Slack and tested the complete flow. After that, I added scheduled automation so the report could be generated and sent automatically. Copy of AWS cost Report to Slack Copy of AWS cost Report to Slack

THE WORKFLOW

AWS Cost → Report → Python → Slack → Automation

Has anyone else faced this problem?

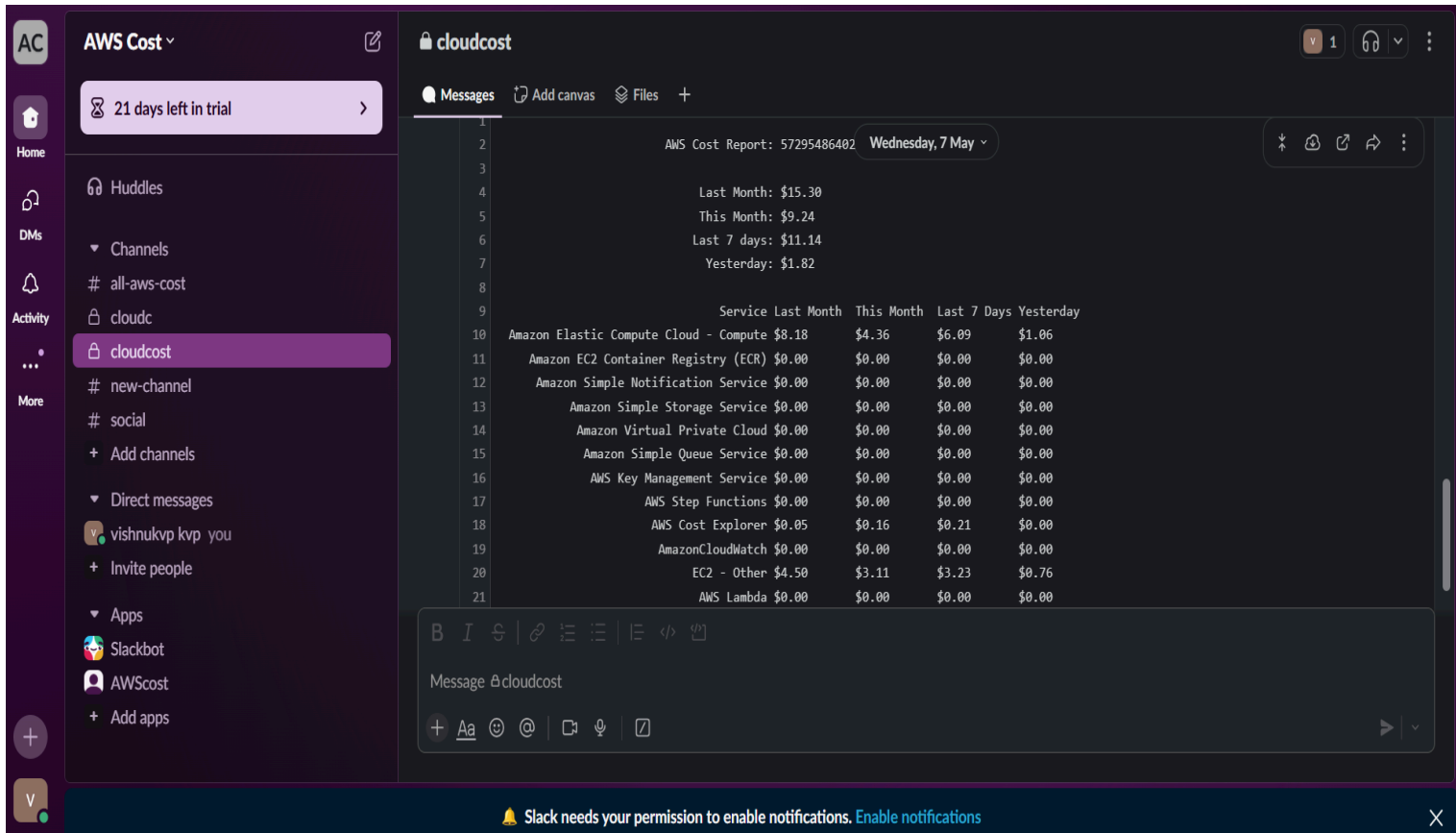
How did you fix it?

Blog:<https://vishnusaik.com>

HashnodeProfile:<https://hashnode.com/@vishnusaik>

GitHub : <https://github.com/ksaivishnusaikvs>

➤ AWS cost notification to Slack channel



cloudcost

21 days left in trial

Messages Add canvas Files +

AWS Cost Report: 57295486402 Wednesday, 7 May

Last Month: \$15.30
This Month: \$9.24
Last 7 days: \$11.14
Yesterday: \$1.82

Service	Last Month	This Month	Last 7 Days	Yesterday
Amazon Elastic Compute Cloud - Compute	\$8.18	\$4.36	\$6.09	\$1.06
Amazon EC2 Container Registry (ECR)	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Notification Service	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Storage Service	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Virtual Private Cloud	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Queue Service	\$0.00	\$0.00	\$0.00	\$0.00
AWS Key Management Service	\$0.00	\$0.00	\$0.00	\$0.00
AWS Step Functions	\$0.00	\$0.00	\$0.00	\$0.00
AWS Cost Explorer	\$0.05	\$0.16	\$0.21	\$0.00
AmazonCloudWatch	\$0.00	\$0.00	\$0.00	\$0.00
EC2 - Other	\$4.50	\$3.11	\$3.23	\$0.76
AWS Lambda	\$0.00	\$0.00	\$0.00	\$0.00

Slack needs your permission to enable notifications. [Enable notifications](#)

➤ To check clearly: select the 3 dots, click on View raw

AWS Cost Report: 572954864028

Last Month: \$15.30
This Month: \$28.93
Last 7 days: \$18.36
Yesterday: \$0.83

Service	Last Month	This Month	Last 7 Days	Yesterday
Amazon Elastic Compute Cloud - Compute	\$8.18	\$16.01	\$12.71	\$0.00
Amazon EC2 Container Registry (ECR)	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Notification Service	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Storage Service	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Virtual Private Cloud	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Queue Service	\$0.00	\$0.00	\$0.00	\$0.00
AWS Key Management Service	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Location Service	\$0.00	\$0.00	\$0.00	\$0.00
AWS Secrets Manager	\$0.00	\$0.00	\$0.00	\$0.00
AWS Service Catalog	\$0.00	\$0.00	\$0.00	\$0.00
AWS Step Functions	\$0.00	\$0.00	\$0.00	\$0.00
AWS Cost Explorer	\$0.05	\$0.17	\$0.01	\$0.00
AmazonCloudWatch	\$0.00	\$0.00	\$0.00	\$0.00
Amazon Glacier	\$0.00	\$0.00	\$0.00	\$0.00
EC2 - Other	\$4.50	\$8.57	\$5.64	\$0.83
AWS Lambda	\$0.00	\$0.00	\$0.00	\$0.00
AWS Glue	\$0.00	\$0.00	\$0.00	\$0.00
Tax	\$2.57	\$4.18	\$0.00	\$0.00

STEP1: Set up an EC2 instance first. Launch the EC2 instance in an AWS region, run apt-get update, and update the packages

Install Node.js from <https://nodejs.org/en/download>, select os and use it. In my case



Installation looks like this.

```
root@ip-172-31-2-229:~# apt-get update
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [2511 kB]
Get:6 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [410 kB]
Get:7 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1201 kB]
Get:8 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [296 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2266 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [346 kB]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [3238 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [577 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [973 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [209 kB]
Fetched 12.4 MB in 3s (4077 kB/s)
Reading package lists... Done
root@ip-172-31-2-229:~# curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 16631 100 16631 0 0 263k 0 --:--:-- --:--:-- --:--:-- 266k
=> Downloading nvm from git to '/root/.nvm'
=> Cloning into '/root/.nvm'...
remote: Enumerating objects: 382, done.
remote: Counting objects: 100% (382/382), done.
remote: Compressing objects: 100% (325/325), done.
remote: Total 382 (delta 43), reused 179 (delta 29), pack-reused 0 (from 0)
Receiving objects: 100% (382/382), 385.06 KiB | 17.50 MiB/s, done.
Resolving deltas: 100% (43/43), done.
* (HEAD detached at FETCH_HEAD)
master
=> Compressing and cleaning up git repository
=> Appending nvm source string to /root/.bashrc
=> Appending bash_completion source string to /root/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
root@ip-172-31-2-229:~# \. "$HOME/.nvm/nvm.sh"
root@ip-172-31-2-229:~# nvm install 22
Downloading and installing node v22.15.0...
Downloading https://nodejs.org/dist/v22.15.0/node-v22.15.0-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v22.15.0 (npm v10.9.2)
```

Step2: Installation. Install the package globally, or you can also use npx

npm install -g aws-cost-cli

(Or)

Docker You can build this Dockerfile using the docker build command and then run a container from the built image. For example:

docker build -t aws-cost-cli.

docker run aws-cost-cli

```
root@ip-172-31-2-229:~# npm install -g aws-cost-cli
npm warn deprecated querystring@0.2.0: The querystring API is considered Legacy. new code should use the URLSearchParams API instead.
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead

added 82 packages in 8s

39 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.3.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.3.0
npm notice To update run: npm install -g npm@11.3.0
npm notice
```

Install the command we need to run

aws-cost: This will show the AWS billing cost

```
ubuntu@ip-172-31-2-229:~$ sudo -i
root@ip-172-31-2-229:~# aws-cost

AWS Cost Report: 572954864028

Last Month: $15.30
This Month: $1.51
Last 7 days: $4.28
Yesterday: $1.51

Service Last Month This Month Last 7 Days Yesterday
Amazon Elastic Compute Cloud - Compute $8.18 $0.69 $2.39 $0.69
Amazon EC2 Container Registry (ECR) $0.00 $0.00 $0.00 $0.00
Amazon Simple Notification Service $0.00 $0.00 $0.00 $0.00
Amazon Simple Storage Service $0.00 $0.00 $0.00 $0.00
Amazon Virtual Private Cloud $0.00 $0.00 $0.00 $0.00
Amazon Simple Queue Service $0.00 $0.00 $0.00 $0.00
AWS Key Management Service $0.00 $0.00 $0.00 $0.00
AWS Step Functions $0.00 $0.00 $0.00 $0.00
AWS Cost Explorer $0.05 $0.00 $0.00 $0.00
AmazonCloudWatch $0.00 $0.00 $0.00 $0.00
EC2 - Other $4.50 $0.82 $1.89 $0.82
AWS Lambda $0.00 $0.00 $0.00 $0.00
Tax $2.57 $0.00 $0.00 $0.00

root@ip-172-31-2-229:~#
```

- `aws-cost --help`, it will help you find commands

```
root@ip-172-31-2-229:~# aws-cost --help
Usage: aws-cost [options]

A CLI tool to perform cost analysis on your AWS account

Options:
  -V, --version                output the version number
  -p, --profile [profile]      AWS profile to use (default: "default")
  -k, --access-key [key]       AWS access key
  -s, --secret-key [key]       AWS secret key
  -T, --session-token [key]    AWS session Token
  -r, --region [region]        AWS region (default: "us-east-1")
  -j, --json                   Get the output as JSON
  -u, --summary                Get only the summary without service breakdown
  -t, --text                   Get the output as plain text (no colors / tables)
  -S, --slack-token [token]    Token for the slack integration
  -C, --slack-channel [channel] Channel to which the slack integration should post
  -h, --help                   Get the help of the CLI
root@ip-172-31-2-229:~# aws-cost --summary

AWS Cost Report: 572954864028

Last Month: $15.02
This Month: $0.00
Last 7 days: $4.28
Yesterday: $1.67
```

How to integrate with Slack

Step1: Install the AWS CLI in your instance

Install AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

```
sudo apt-get install unzip -y
```

```
unzip awscliv2.zip
```

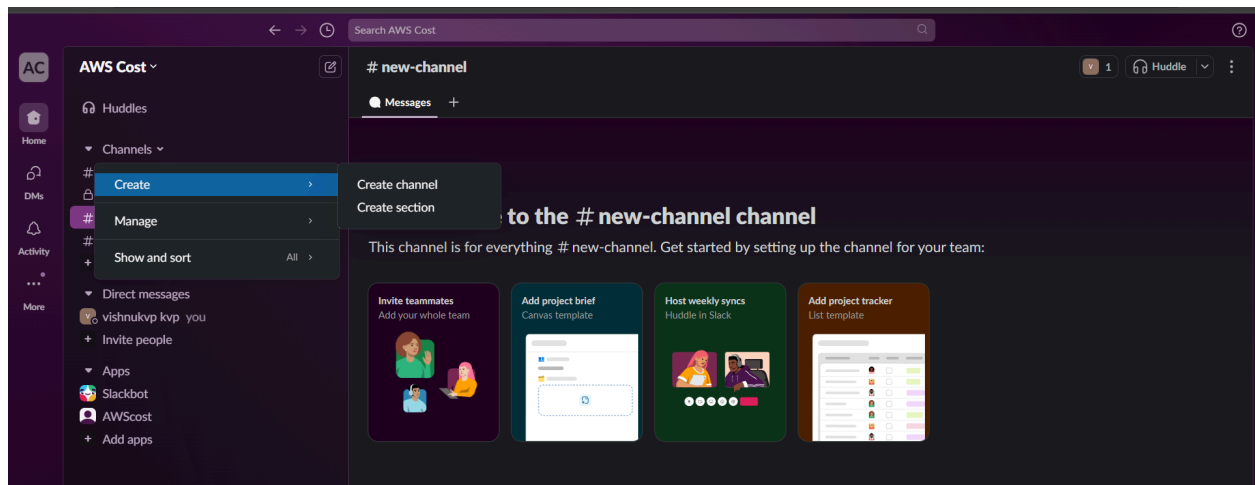
```
sudo ./aws/install
```

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

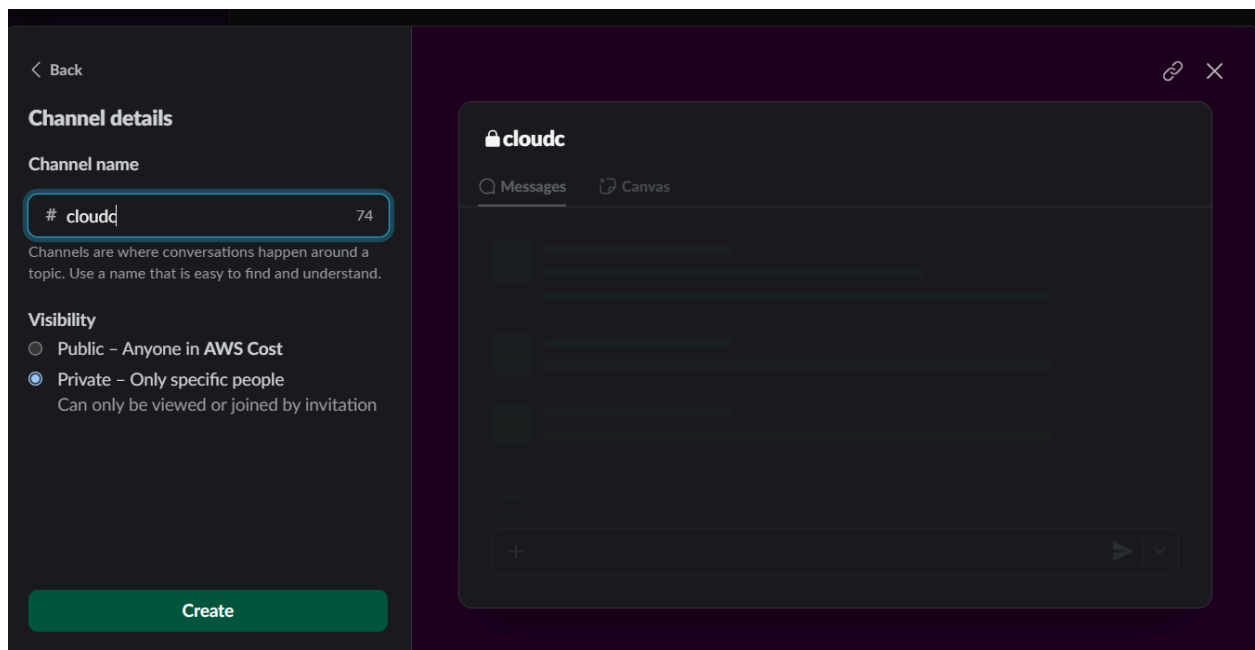
Try the command

AWS configure or IAM roles.

Step2: Go to Slack —> first we need to create a workspace, then create the channel → click on Next



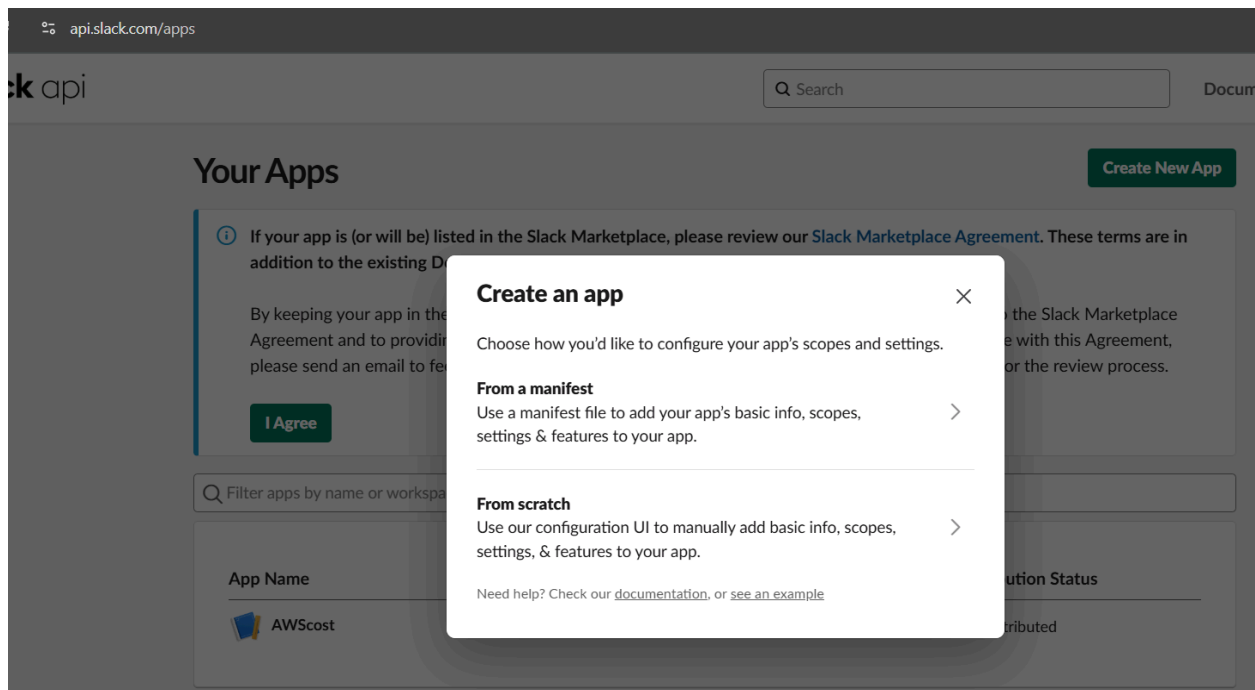
Step3: Give the channel a name → select the visibility (public or private best practices



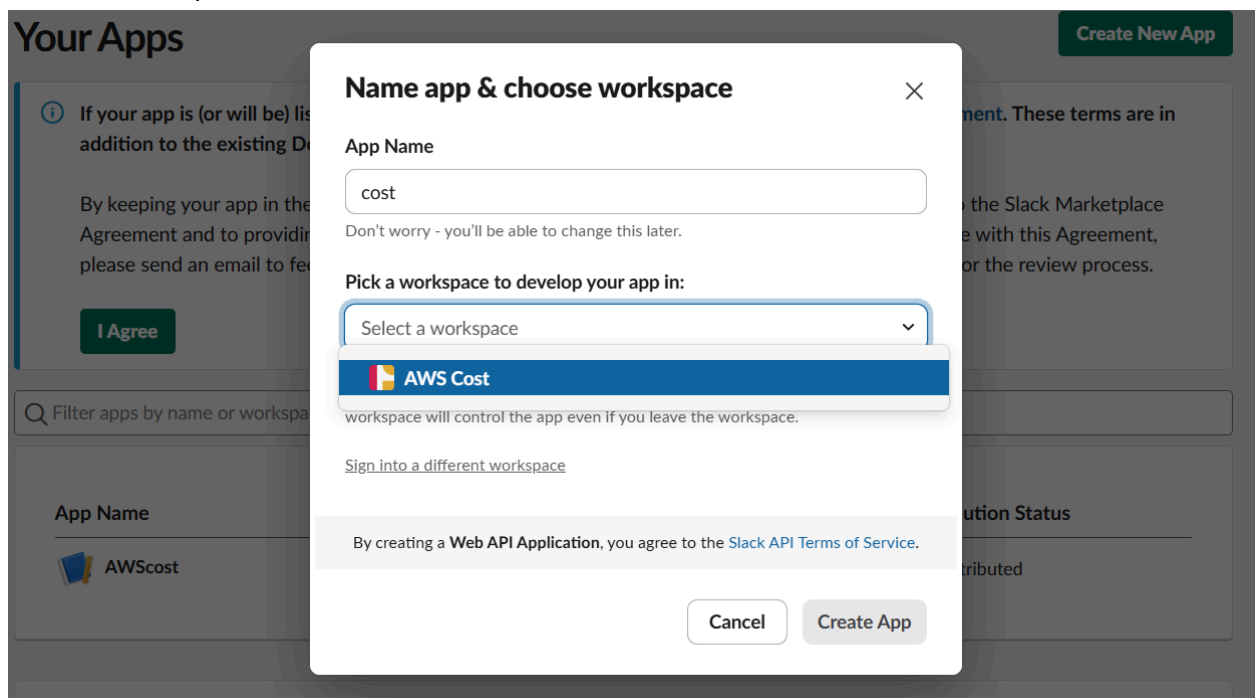
Click on Create the channel • click Create

Step4:

Go to <https://api.slack.com/apps> → click on the new app → select from scratch



Step 5: Give your app a name and select the workspace in Slack. If no workspace is created, create a workspace.



Once it is created, it looks like this.

App Credentials

These credentials allow your app to access the Slack API. They are secret. Please don't share your app credentials with anyone, include them in public code repositories, or store them in insecure ways.

App ID

A08QYJ503A6

Date of App Creation

May 4, 2025

Client ID

8844374581409.8848617003346

Client Secret

••••••••••

Show

Regenerate

You'll need to send this secret along with your client ID when making your [oauth.v2.access](#) request.

Signing Secret

••••••••••

Show

Regenerate

Slack signs the requests we send you using this secret. Confirm that each request comes from Slack by verifying its unique signature.

Verification Token

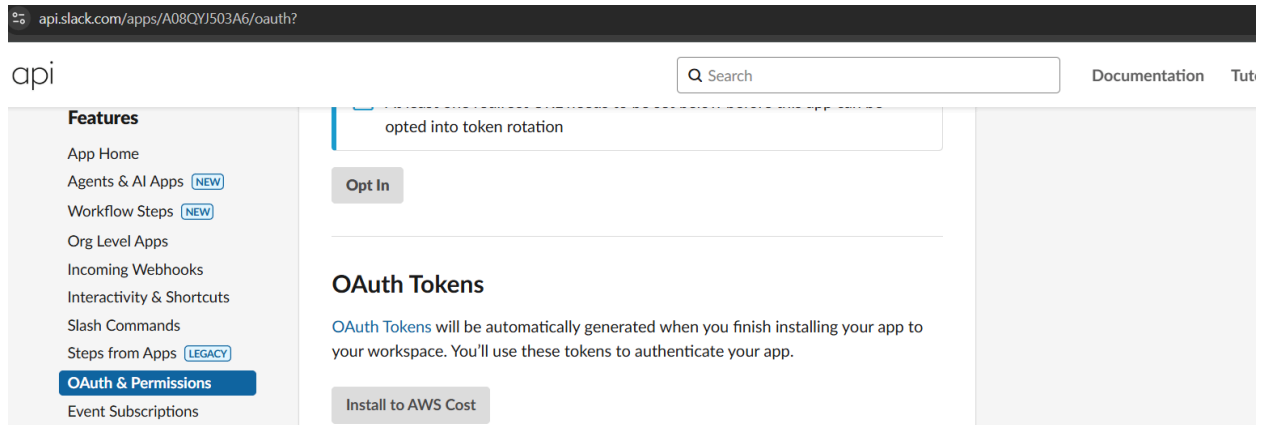
mtSnOiPz8YNPDCA7GQBP87WT

Regenerate

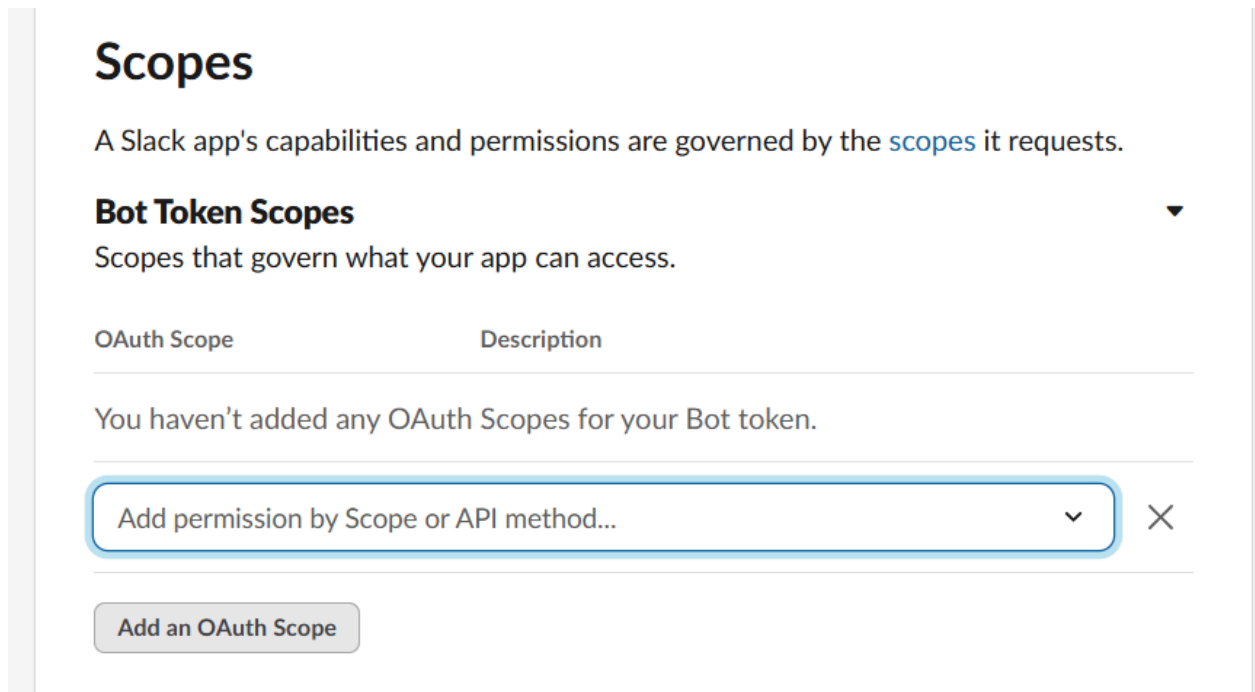
Step6:

The image text describes how to set Slack app permissions in the app dashboard:

Go to "OAuth & Permissions".



Scroll to "Scopes".



Under "Bot Token Scopes", add:

chat.write → Allows the bot to post messages.

chat.write.public → (Optional) Allows posting in public channels without being invited.

Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.

Bot Token Scopes

Scopes that govern what your app can access.

OAuth Scope	Description	
chat:write	Send messages as cost	

chat

chat:write.customize

Send messages as cost with a customized username and avatar

chat:write.public

Send messages to channels cost isn't a member of

links:write

Show previews of URLs in messages

OAuth Scope	Description
-------------	-------------

You haven't added any OAuth Scopes for your User token.

Add an OAuth Scope

Once they add all

Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.

Bot Token Scopes

Scopes that govern what your app can access.

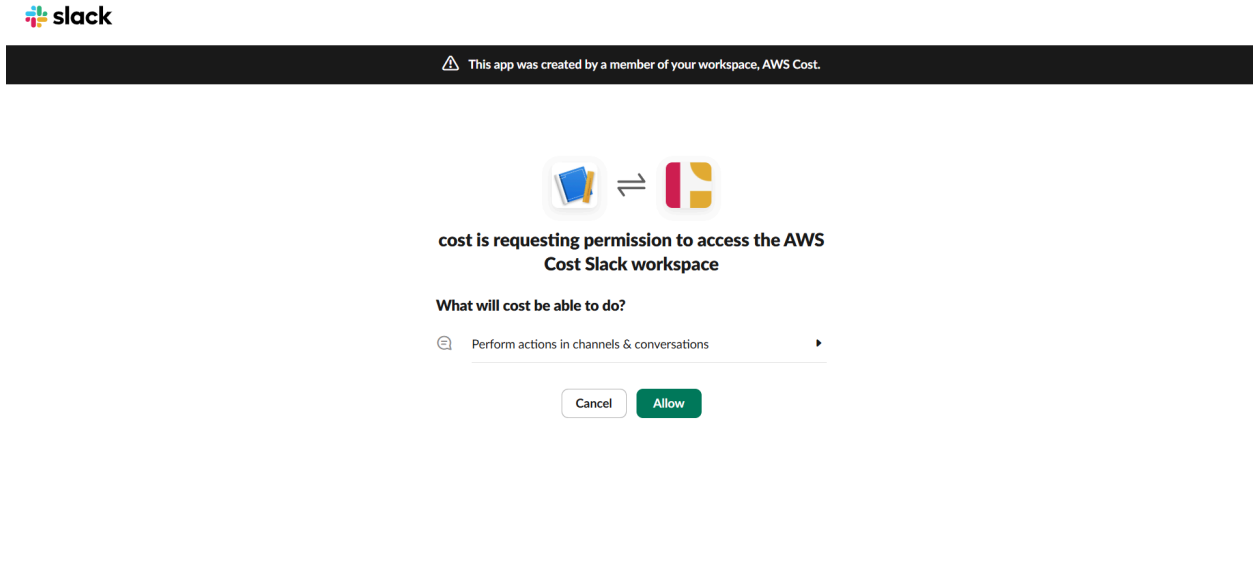
OAuth Scope	Description	
chat:write	Send messages as cost	
chat:write.public	Send messages to channels cost isn't a member of	

Add an OAuth Scope

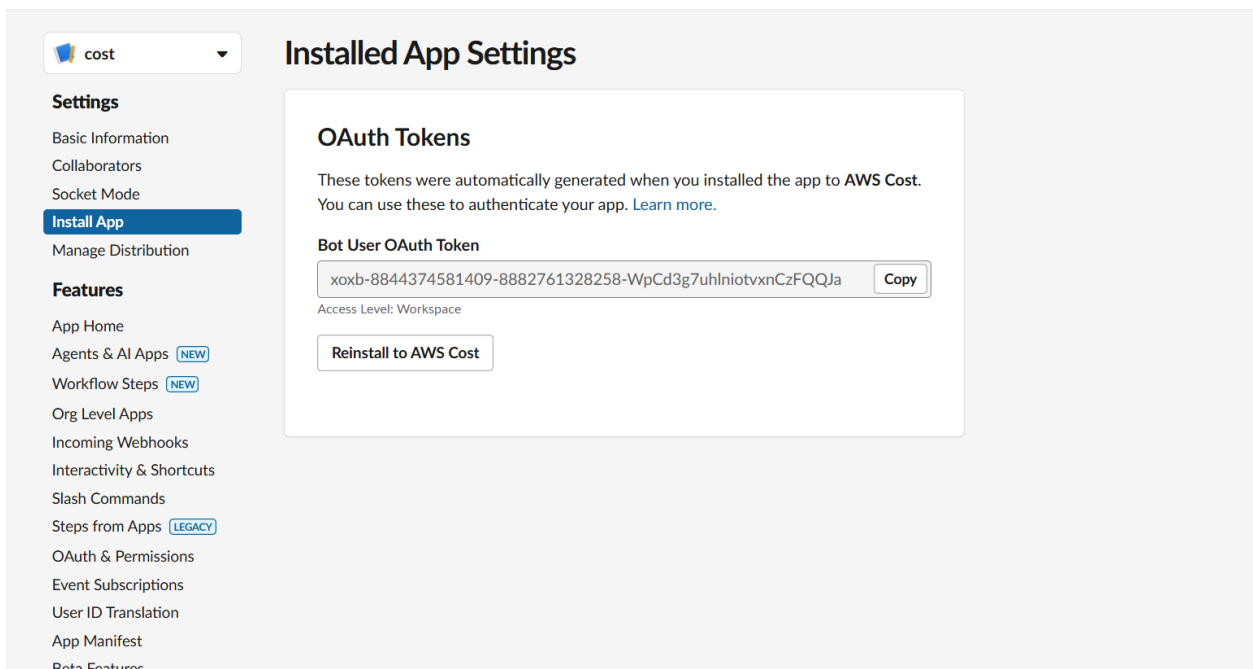
Step 7: Install the App in Your Workspace

1. Still in OAuth & Permissions, click Install App to Workspace.
2. Approve the permissions.
3. Copy the Bot User OAuth Token:
 - Starts with xoxb-... → You'll need this for aws-cost-cli.

Approve the permissions.



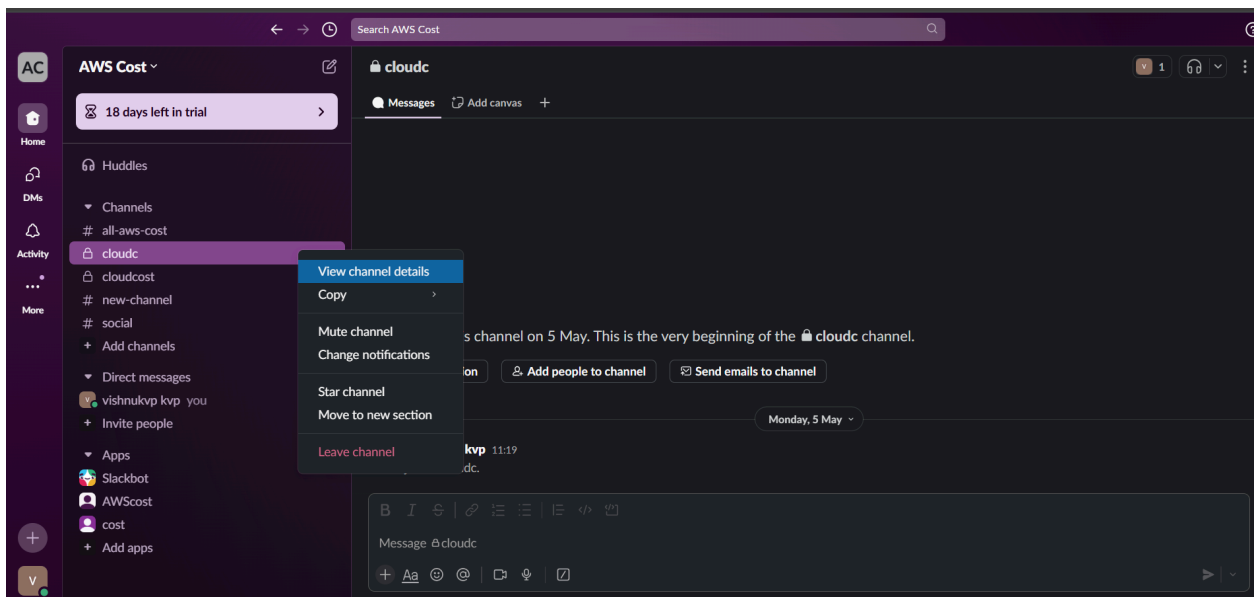
Then we need to see the token; it starts with xoxb-... → You'll need this for aws-cost-cli.



step8:

Find Your Slack Channel ID

1. Open Slack.
2. Go to the channel you want to post in.
3. Click on the channel name at the top → View channel details.
4. Copy the Channel ID (starts with C...).





Enable notifications



Huddle

About

Members 1

Tabs

Integrations

Settings

Description

Add a description

Managed by ⓘ

vishnukvp kvp

Created by

vishnukvp kvp on 5 May 2025

Leave channel

Files

There aren't any files to see here right now. But there could be – drop any file into the message pane to add it to this conversation.

Channel ID: C08QFUC571D

Step9: Test the Slack integration with aws-cost-cli

```
aws-cost --slack-token xoxb-884434558977825-12ZAU3PI9zCCkJyeQH5oK8tO --slack-channel C08QKDNcw59
```

Change the token and channel name.

```
root@ip-172-31-2-229:~# aws-cost --slack-token xoxb-8844374581409-8844558977825-12ZAU3P19zCCkJyeQH5oK8tO --slack-channel C08QKDNcw59

AWS Cost Report: 572954864028

Last Month: $15.30
This Month: $28.93
Last 7 days: $18.36
Yesterday: $0.83

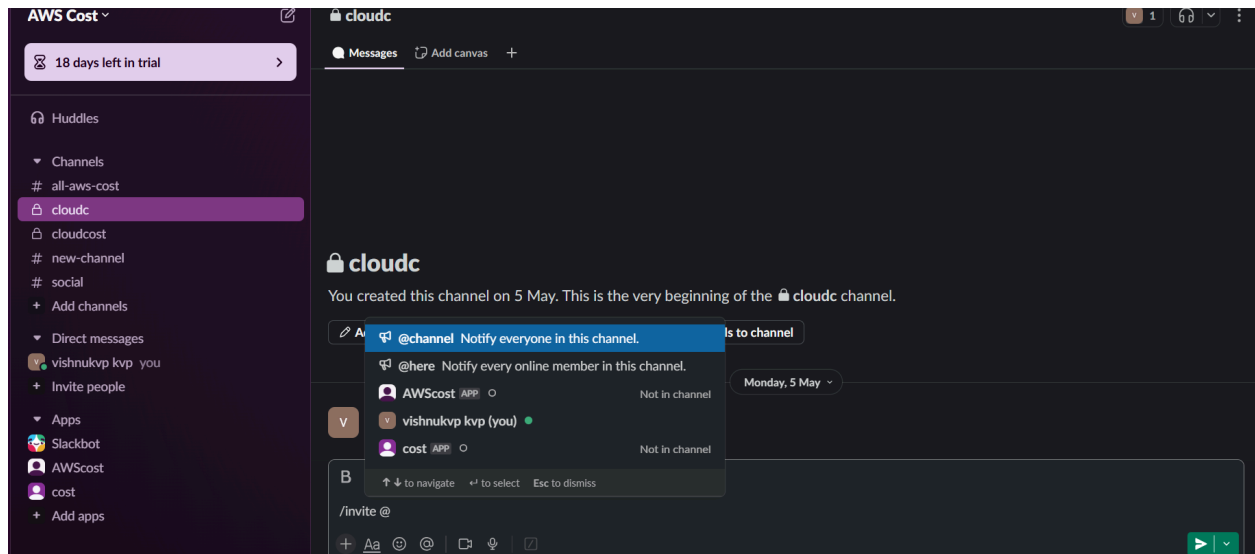
Service Last Month This Month Last 7 Days Yesterday
Amazon Elastic Compute Cloud - Compute $8.18 $16.01 $12.71 $0.00
Amazon EC2 Container Registry (ECR) $0.00 $0.00 $0.00 $0.00
Amazon Simple Notification Service $0.00 $0.00 $0.00 $0.00
Amazon Simple Storage Service $0.00 $0.00 $0.00 $0.00
Amazon Virtual Private Cloud $0.00 $0.00 $0.00 $0.00
Amazon Simple Queue Service $0.00 $0.00 $0.00 $0.00
AWS Key Management Service $0.00 $0.00 $0.00 $0.00
Amazon Location Service $0.00 $0.00 $0.00 $0.00
AWS Secrets Manager $0.00 $0.00 $0.00 $0.00
AWS Service Catalog $0.00 $0.00 $0.00 $0.00
AWS Step Functions $0.00 $0.00 $0.00 $0.00
AWS Cost Explorer $0.05 $0.17 $0.01 $0.00
AmazonCloudwatch $0.00 $0.00 $0.00 $0.00
Amazon Glacier $0.00 $0.00 $0.00 $0.00
EC2 - Other $4.50 $8.57 $5.64 $0.83
AWS Lambda $0.00 $0.00 $0.00 $0.00
AWS Glue $0.00 $0.00 $0.00 $0.00
Tax $2.57 $4.18 $0.00 $0.00

Successfully sent message to Slack
root@ip-172-31-2-229:~#
```

Verify that we received the message on Slack.

Note: If your Slack invitation fails to send a message to the Slack channel not found, we need to invert the process for the first time (or) failed Slack not found

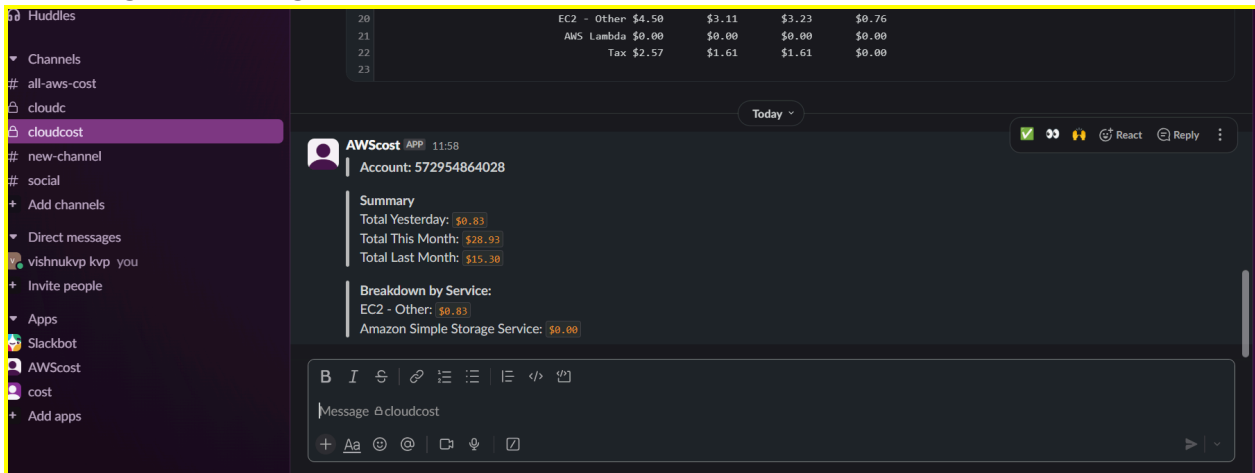
Step 9.1: `/invite @ your app name that shows in the image`



Run the command

```
aws-cost --slack-token xoxb-8844374581409-8844558977825-12ZAU3PI9zCCkJyeQH5oK8tO
--slack-channel C08QKDNCW59
```

Now we get a message on Slack



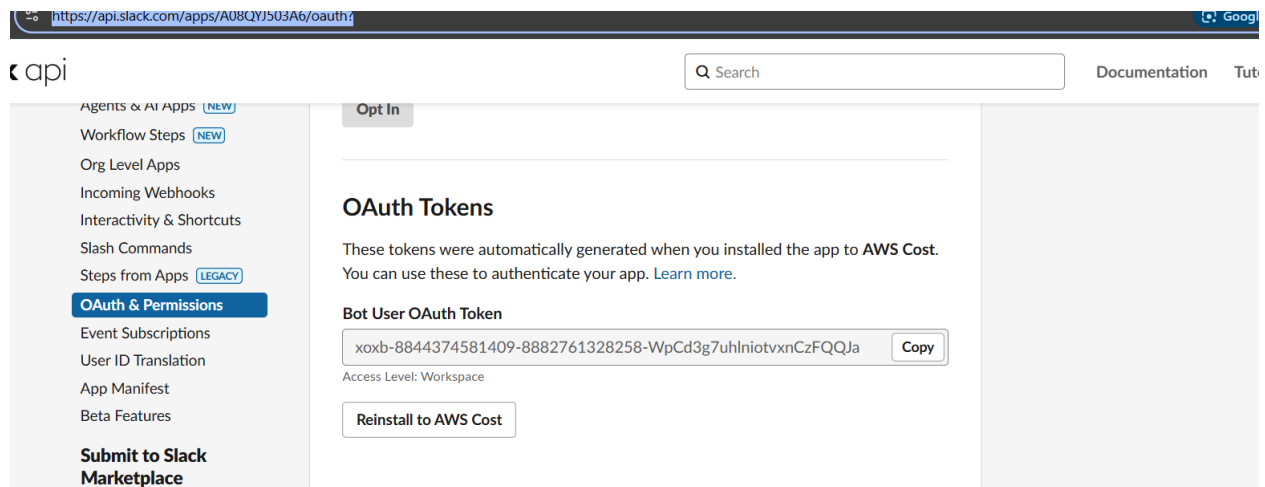
This output is not clear format. We need to print the file format.

Step10: /invite @AWS-COST-Report

```
aws-cost --text > cost-report.txt && \  
curl -F file=@cost-report.txt \  
  -F "initial_comment=AWS Cost Report" \  
  -F channels=C08PFVZV9PF \  
  -H "Authorization: Bearer \  
xoxb-6800278956247-8805347127382-SQlckF4aKrzil8ODyqARQRA" \  
https://slack.com/api/files.upload
```

This command generates an AWS cost report, saves it to a text file, and then uploads it to a **Slack channel** using Slack's API.

Step10.1: We need to print in text format. We need to add the go for the <https://api.slack.com/apps/A08QYJ503A6/oauth?>

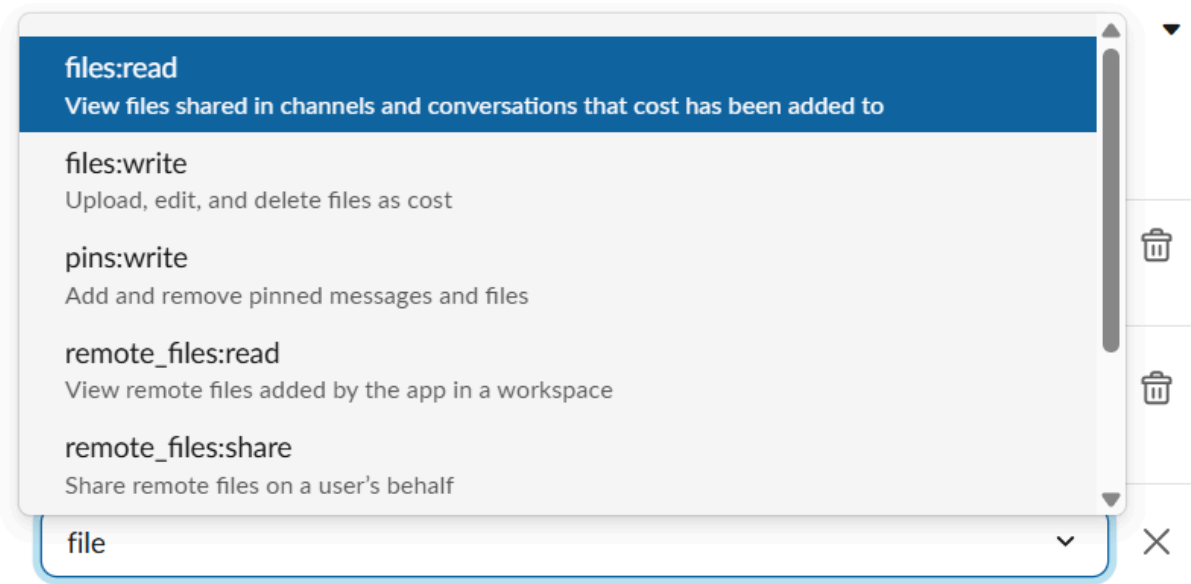


The screenshot shows the Slack API interface for the 'AWS Cost' app (A08QYJ503A6). The left sidebar contains a navigation menu with options like 'Agents & AI Apps', 'Workflow Steps', 'Org Level Apps', 'Incoming Webhooks', 'Interactivity & Shortcuts', 'Slash Commands', 'Steps from Apps', 'OAuth & Permissions' (highlighted), 'Event Subscriptions', 'User ID Translation', 'App Manifest', 'Beta Features', and 'Submit to Slack Marketplace'. The main content area is titled 'OAuth Tokens' and includes a search bar, 'Documentation', and 'Tutorials' links. It states: 'These tokens were automatically generated when you installed the app to AWS Cost. You can use these to authenticate your app. [Learn more.](#)' Under 'Bot User OAuth Token', a text box displays the token 'xoxb-8844374581409-8882761328258-WpCd3g7uhlniotvxnCzFQQJa' with a 'Copy' button. Below the token, it shows 'Access Level: Workspace' and a 'Reinstall to AWS Cost' button.

We need to print the file format in Slack. We add on the go→ scope → add the file write.

Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.



Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.

Bot Token Scopes

Scopes that govern what your app can access.

OAuth Scope	Description	
chat:write	Send messages as @cost	
chat:write.public	Send messages to channels @cost isn't a member of	
files:write	Upload, edit, and delete files as cost	

After adding the file, write in the scope we need to reinstall the app, click on the reinstall app in top.



DocumentationTutorialsYour Apps

⚠ You've changed the permission scopes your app uses. Please [reinstall your app](#) for these changes to take effect (and if your app is listed in the Slack Marketplace, you'll need to resubmit it as well).

Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.

Bot Token Scopes

Scopes that govern what your app can access.

OAuth Scope	Description	
chat:write	Send messages as @cost	
chat:write.public	Send messages to channels @cost isn't a member of	
files:write	Upload, edit, and delete files as cost	

[Add an OAuth Scope](#)

User Token Scopes

Scopes that access user data and act on behalf of users that authorize them

Click on Allow

⚠ This app was created by a member of your workspace, AWS Cost.

cost is requesting permission to access the AWS Cost Slack workspace

What will cost be able to do?

 Perform actions in channels & conversations

[Cancel](#) [Allow](#)

Step 11: Install Python & pip (if not already)

sudo apt update

sudo apt install -y python3 python3-pip python3-venv

2. Create a Virtual Environment (Optional but Recommended)

python3 -m venv venv

source venv/bin/activate

Step12: We install

pip install slack-sdk

2. Create Python Script (`upload_cost_report.py`):

```
from slack_sdk import WebClient
```

```
from slack_sdk.errors import SlackApiError
```

```
# Your Slack Bot Token
```

```
slack_token = "xoxb-6800278956247-DyqARQRA"
```

```
client = WebClient(token=slack_token)
```

```
try:
```

```
    # Upload the file
```

```
    response = client.files_upload(
```

```
        channels="C08PFVZV9PF",
```

```
        file="cost-report.txt",
```

```
        initial_comment="AWS Cost Report"
```

```
    )
```

```
    print("File uploaded successfully:", response)
```

```
except SlackApiError as e:
```

```
    print(f"Error uploading file: {e.response['error']}")
```

```
root@ip-172-31-2-229:~# cat aws_cost_slack.py
from slack_sdk import WebClient
from slack_sdk.errors import SlackApiError

# Your Slack Bot Token
slack_token = "xoxb-8844374581409-8844558977825-12ZAU3P19zCCKJyeQH5ok8to"
client = WebClient(token=slack_token)

try:
    with open("cost-report.txt", "rb") as f:
        response = client.files_upload_v2(
            channel="C08QKDNW59",
            file=f,
            filename="cost-report.txt",
            title="AWS Cost Report",
            initial_comment="AWS Cost Report"
        )
    print("File uploaded successfully:", response)
except SlackApiError as e:
    print(f"Error uploading file: {e.response['error']}")

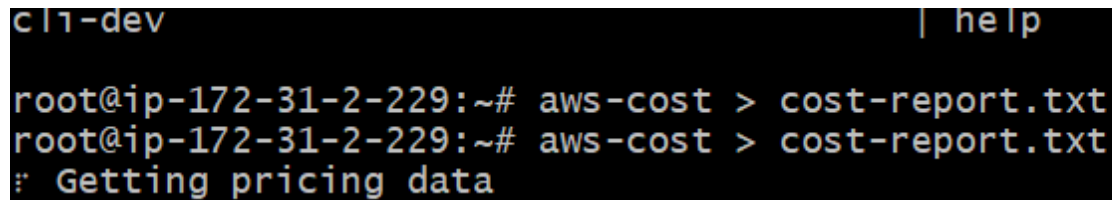
root@ip-172-31-2-229:~# |
```

Step12: creat file vi aws_cost_slack.py, add script give the executed permissions

Run the AWS Cost CLI + Python Script:

Generate cost report

```
aws-cost --text > cost-report.txt
```

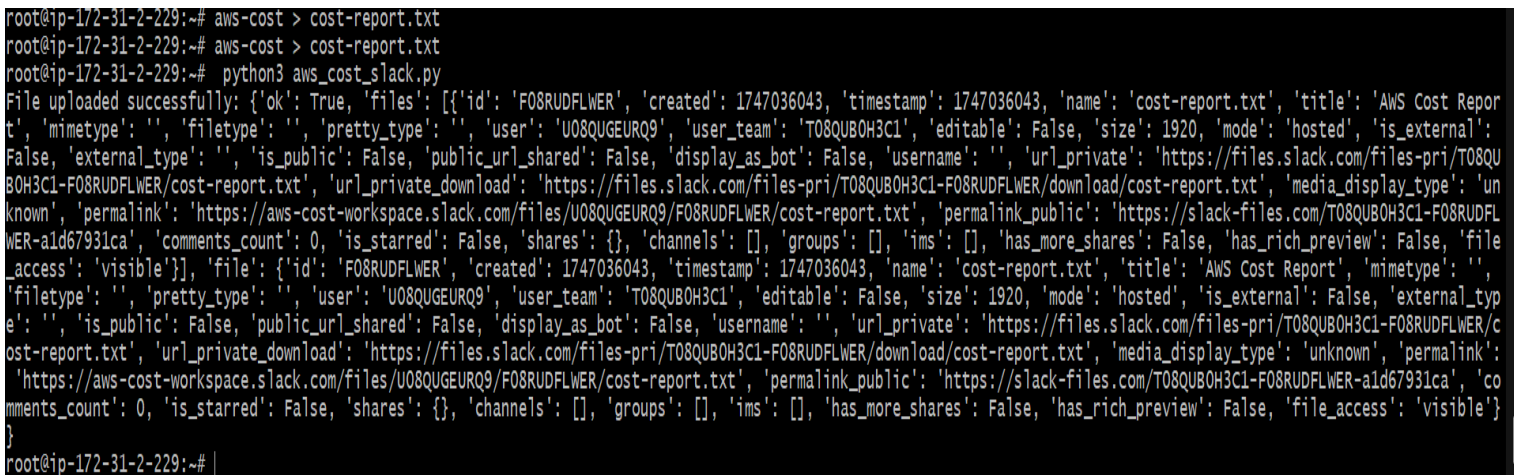


```
cli-dev | help  
root@ip-172-31-2-229:~# aws-cost > cost-report.txt  
root@ip-172-31-2-229:~# aws-cost > cost-report.txt  
: Getting pricing data
```

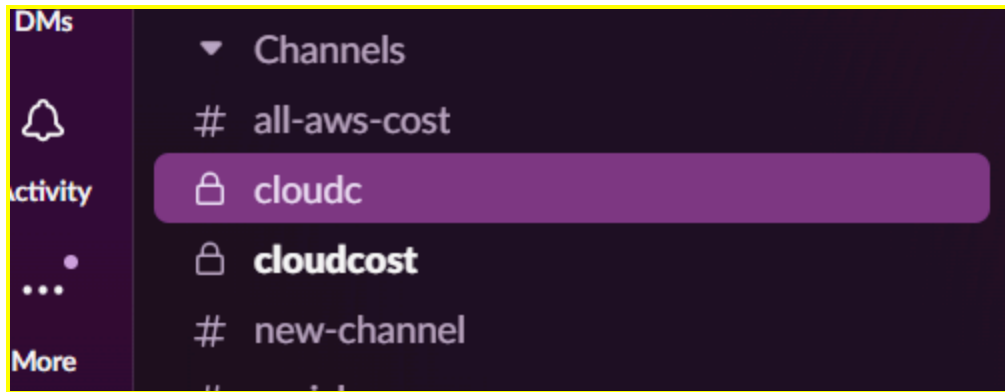
Every time we run the command, we run the script; it gets a new report to text.

Upload to Slack

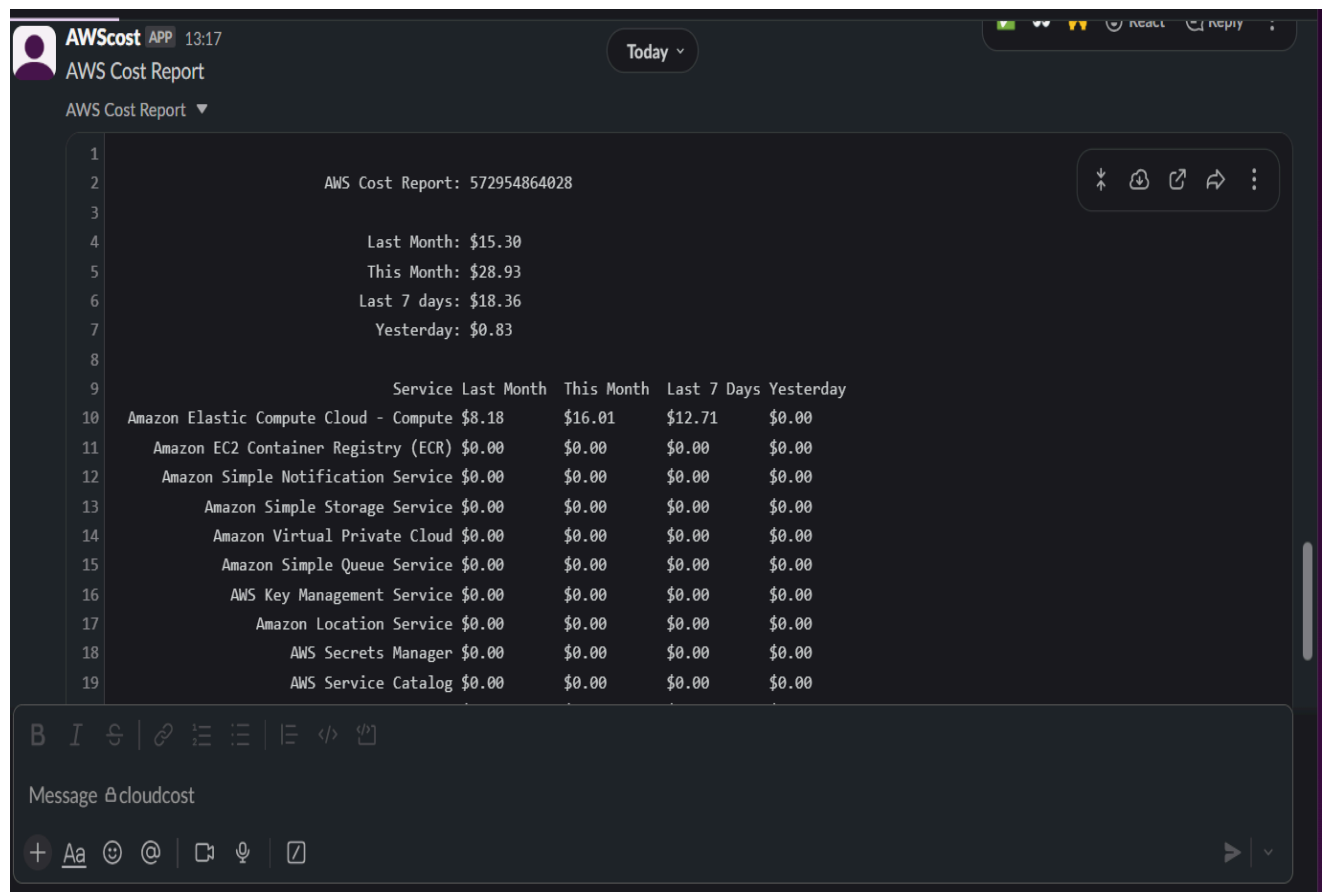
```
python3 upload_cost_report.py
```



```
root@ip-172-31-2-229:~# aws-cost > cost-report.txt  
root@ip-172-31-2-229:~# aws-cost > cost-report.txt  
root@ip-172-31-2-229:~# python3 aws_cost_slack.py  
File uploaded successfully: {'ok': True, 'files': [{'id': 'F08RUDFLWER', 'created': 1747036043, 'timestamp': 1747036043, 'name': 'cost-report.txt', 'title': 'AWS Cost Report', 'mimetype': '', 'filetype': '', 'pretty_type': '', 'user': 'U08QUGEURQ9', 'user_team': 'T08QUB0H3C1', 'editable': False, 'size': 1920, 'mode': 'hosted', 'is_external': False, 'external_type': '', 'is_public': False, 'public_url_shared': False, 'display_as_bot': False, 'username': '', 'url_private': 'https://files.slack.com/files-pri/T08QUB0H3C1-F08RUDFLWER/cost-report.txt', 'url_private_download': 'https://files.slack.com/files-pri/T08QUB0H3C1-F08RUDFLWER/download/cost-report.txt', 'media_display_type': 'unknown', 'permalink': 'https://aws-cost-workspace.slack.com/files/U08QUGEURQ9/F08RUDFLWER/cost-report.txt', 'permalink_public': 'https://slack-files.com/T08QUB0H3C1-F08RUDFLWER-a1d67931ca', 'comments_count': 0, 'is_starred': False, 'shares': {}, 'channels': [], 'groups': [], 'ims': [], 'has_more_shares': False, 'has_rich_preview': False, 'file_access': 'visible'}], 'file': {'id': 'F08RUDFLWER', 'created': 1747036043, 'timestamp': 1747036043, 'name': 'cost-report.txt', 'title': 'AWS Cost Report', 'mimetype': '', 'filetype': '', 'pretty_type': '', 'user': 'U08QUGEURQ9', 'user_team': 'T08QUB0H3C1', 'editable': False, 'size': 1920, 'mode': 'hosted', 'is_external': False, 'external_type': '', 'is_public': False, 'public_url_shared': False, 'display_as_bot': False, 'username': '', 'url_private': 'https://files.slack.com/files-pri/T08QUB0H3C1-F08RUDFLWER/cost-report.txt', 'url_private_download': 'https://files.slack.com/files-pri/T08QUB0H3C1-F08RUDFLWER/download/cost-report.txt', 'media_display_type': 'unknown', 'permalink': 'https://aws-cost-workspace.slack.com/files/U08QUGEURQ9/F08RUDFLWER/cost-report.txt', 'permalink_public': 'https://slack-files.com/T08QUB0H3C1-F08RUDFLWER-a1d67931ca', 'comments_count': 0, 'is_starred': False, 'shares': {}, 'channels': [], 'groups': [], 'ims': [], 'has_more_shares': False, 'has_rich_preview': False, 'file_access': 'visible'}}  
root@ip-172-31-2-229:~# |
```



After the run, we receive the notification in the channel.



To check clearly: select the 3 dots, click on the view raw

AWS Cost Report: 572954864028				
Last Month: \$15.30				
This Month: \$28.93				
Last 7 days: \$18.36				
Yesterday: \$0.83				
	Service	Last Month	This Month	Last 7 Days Yesterday
Amazon Elastic Compute Cloud - Compute		\$8.18	\$16.01	\$12.71 \$0.00
Amazon EC2 Container Registry (ECR)		\$0.00	\$0.00	\$0.00 \$0.00
Amazon Simple Notification Service		\$0.00	\$0.00	\$0.00 \$0.00
Amazon Simple Storage Service		\$0.00	\$0.00	\$0.00 \$0.00
Amazon Virtual Private Cloud		\$0.00	\$0.00	\$0.00 \$0.00
Amazon Simple Queue Service		\$0.00	\$0.00	\$0.00 \$0.00
AWS Key Management Service		\$0.00	\$0.00	\$0.00 \$0.00
Amazon Location Service		\$0.00	\$0.00	\$0.00 \$0.00
AWS Secrets Manager		\$0.00	\$0.00	\$0.00 \$0.00
AWS Service Catalog		\$0.00	\$0.00	\$0.00 \$0.00
AWS Step Functions		\$0.00	\$0.00	\$0.00 \$0.00
AWS Cost Explorer		\$0.05	\$0.17	\$0.01 \$0.00
AmazonCloudWatch		\$0.00	\$0.00	\$0.00 \$0.00
Amazon Glacier		\$0.00	\$0.00	\$0.00 \$0.00
EC2 - Other		\$4.50	\$8.57	\$5.64 \$0.83
AWS Lambda		\$0.00	\$0.00	\$0.00 \$0.00
AWS Glue		\$0.00	\$0.00	\$0.00 \$0.00
Tax		\$2.57	\$4.18	\$0.00 \$0.00

Option 1: Using cron (on Linux/Ubuntu EC2)

Step-by-step:

Open crontab:

```
crontab -e
```

Add a cron job to run the script every day at 8 AM (or your preferred time):

```
0 8 * * * /usr/bin/aws-cost --text > /root/cost-report.txt &&  
/usr/bin/python3 /root/aws_cost_slack.py
```

- Adjust paths if `aws-cost` or `python3` are in different locations.
- Use `which aws-cost` and `which python3` to find full paths.

Option 2: Using a Shell Script + cron

Create a shell script:

```
aws_daily_report.sh:
```

```
#!/bin/bash  
/usr/bin/aws-cost --text > /root/cost-report.txt  
/usr/bin/python3 /root/aws_cost_slack.py
```

1. Make it executable:

```
chmod +x aws_daily_report.sh
```

Schedule it via cron:

```
crontab -e
```

```
0 8 * * * /root/aws_daily_report.sh
```

Option 3: AWS Lambda + CloudWatch (serverless, scalable)

Use AWS Lambda to:

- Generate the report
- Upload it to Slack
- Schedule with **CloudWatch Events**

This is better for **multi-user orgs** or **100+ team-wide notifications**.

If you're interested, I can generate an **end-to-end Lambda version** of your script.

One-Time Setup

You first run this **one-time command** to get daily AWS cost data grouped by service for a **past month** (April 2024):

```
aws ce get-cost-and-usage \
  --time-period Start=2024-04-01,End=2024-04-30 \
  --granularity DAILY \
  --metrics "UnblendedCost" \
  --group-by Type=SERVICE \
  --output table
```

This is a **historical report**.

Breakdown

alias aws-cost="..."

Creates a **command shortcut**. After this, you can just type **aws-cost** in your terminal, and it will run the entire AWS CLI command.

aws ce get-cost-and-usage

This is the AWS CLI command for **Cost Explorer**, used to fetch billing and usage data.

- **ce** = Cost Explorer
- **get-cost-and-usage** = Retrieves AWS cost data

--time-period Start=...,End=...

Specifies the **date range** of the cost report.

Start=\$(date -d 'yesterday' +%Y-%m-%d)

End=\$(date +%Y-%m-%d)

- **Start** = **yesterday's date** (e.g., 2025-05-23)
- **End** = **today's date** (e.g., 2025-05-24)

This means: **Get costs for yesterday**.

--granularity DAILY

This tells AWS to give you cost data **broken down per day**. Since you're only asking for one day (**yesterday**), it gives the cost just for that one day.

♦ **--metrics UnblendedCost**

Specifies the cost metric you want:

- **UnblendedCost** = Actual cost before discounts or credits are applied.

Other options (not used here): **BlendedCost**, **UsageQuantity**.

♦ `--group-by Type=SERVICE`

This groups the output by **AWS service**, so you can see how much each service (like EC2, S3, etc.) costs on that day.

♦ `--output table`

This formats the output in a **readable text table** format.

Alternatives:

- `json` (for scripting)
- `text`
- `csv`

Example Output:

When you run:

```
aws-cost
```

```
+-----+-----+
| Service                               | Amount   |
+-----+-----+
| Amazon Elastic Compute Cloud - EC2   | $1.11    |
| Amazon S3                             | $0.04    |
+-----+-----+
```

Summary

Option	Purpose
<code>alias</code> <code>aws-cost=...</code>	Creates a shortcut
<code>Start=\$(...)</code>	Uses yesterday's date
<code>--granularity</code> <code>DAILY</code>	Gives daily cost details
<code>--group-by</code> <code>SERVICE</code>	Breaks are cut down by the service
<code>--output table</code>	Human-readable report

Dynamic command

```
alias aws-cost="aws ce get-cost-and-usage \  
--time-period Start=$(date -d 'yesterday' +%Y-%m-%d),End=$(date +%Y-%m-%d) \  
--granularity DAILY \  
--metrics UnblendedCost \  
--group-by Type=SERVICE \  
--output table"
```

It is dynamic, and here's what it does:

- **Start=\$(date -d 'yesterday' +%Y-%m-%d)**
Gets **yesterday's date** in the format **YYYY-MM-DD**.
- **End=\$(date +%Y-%m-%d)**
Gets **today's date**, so the report ends at midnight today.
- **--granularity DAILY**
Ensures the data is broken down **by day**.
- **--group-by Type=SERVICE**
Breaks down cost **by AWS service** (as shown in your screenshot).
- **--output table**
Formats the output in a **readable text table** (like your screenshot).

Result

Each time you run:

```
aws-cost > cost-report.txt
```

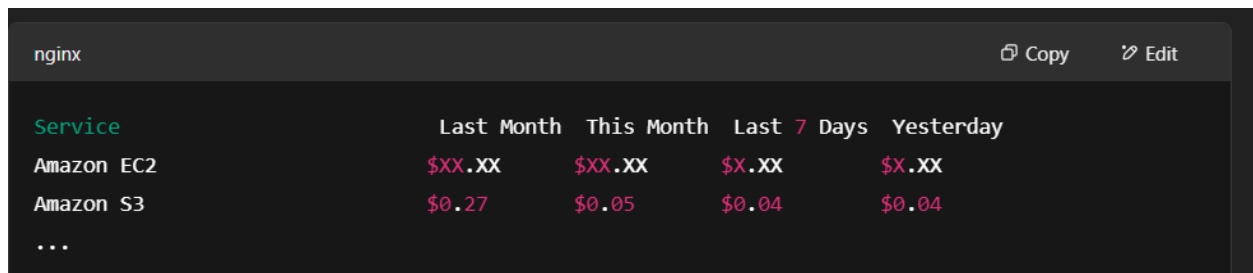
It will dynamically generate a **yesterday-only report** for all AWS services and store it in **cost-report.txt**, in the same **table format** shown in your screenshot.

Example Output (based on your screenshot):

Each day, you'll get something like:

nginx

Service	Last Month	This Month	Last 7 Days
Yesterday			
Amazon EC2	\$XX.XX	\$XX.XX	\$X.XX
\$X.XX			
Amazon S3	\$0.27	\$0.05	\$0.04
\$0.04			



Service	Last Month	This Month	Last 7 Days	Yesterday
Amazon EC2	\$XX.XX	\$XX.XX	\$X.XX	\$X.XX
Amazon S3	\$0.27	\$0.05	\$0.04	\$0.04
...				

Summary

- Yes, your **AWS cost** alias **will dynamically generate** this report **every day** for **yesterday**.
- The output is formatted as a readable **text table**.
- You can automate or script it for Slack, email, or log files.

Let me know if you want to:

- Convert it to JSON/CSV for scripting.
- Filter to only non-zero services.
- Format it for Slack better.

Summary

- Yes, your **AWS cost** alias **will dynamically generate** this report **every day** for **yesterday**.
- The output is formatted as a readable **text table**.
- You can automate or script it for Slack, email, or log files.

Let me know if you want to:

- Convert it to JSON/CSV for scripting.
- Filter to only non-zero services.
- Format it for Slack better.

Step 1: Modify Your **AWS-Cost** Alias to Output JSON

You'll use **JSON output** (easier to process programmatically):

```
alias aws-cost="aws ce get-cost-and-usage \
  --time-period Start=$(date -d 'yesterday' +%Y-%m-%d),End=$(date
+%Y-%m-%d) \
  --granularity DAILY \
  --metrics UnblendedCost \
  --group-by Type=SERVICE \
  --output json"
```

Add this alias to your `~/.bashrc` or `~/.zshrc`, then run `source ~/.bashrc` (or equivalent shell).

Step 2: Run the Alias & Save Output

```
aws-cost > cost-report.json
```

Modified Command for CSV Output:

```
alias aws-cost="aws ce get-cost-and-usage \
  --time-period Start=$(date -d 'yesterday'
+%Y-%m-%d),End=$(date +%Y-%m-%d) \
  --granularity DAILY \
  --metrics UnblendedCost \
  --group-by Type=SERVICE \
  --output text | tr '\t' ',' > cost-report.csv"
```

How This Works:

```
--output text
```

This produces output in **plain tab-delimited text** (not real CSV yet).

```
tr '\t' ','
```

This converts **tabs to commas**, making it **CSV-compatible**.

```
> cost-report.csv
```

Redirects the output into a file.

Result:

When you run: `aws-cost`

It will generate a file `cost-report.csv` with a format like:

```
yaml
```

```
CopyEdit
```

```
2025-05-23, Amazon Elastic Compute Cloud - Compute,1.15, USD
```

```
2025-05-23,Amazon S3,0.04,USD
```

Notes:

- AWS CLI does **not** directly support `--output csv`.
- Using `--output text | tr '\t' ','` is the most reliable workaround.
- If you need proper headers or processing, use Python to post-process the file.

Great! Here's a complete **step-by-step guide** to:

1. Get AWS cost data dynamically in **JSON or CSV**
2. Filter for **non-zero** cost services
3. Format it nicely for **Slack messages**
4. Automate if needed

Step 1: Modify Your **AWS-Cost** Alias to Output JSON

You'll use **JSON output** (easier to process programmatically):

bash

CopyEdit

```
alias aws-cost="aws ce get-cost-and-usage \
    --time-period Start=$(date -d 'yesterday'
+%Y-%m-%d),End=$(date +%Y-%m-%d) \
    --granularity DAILY \
    --metrics UnblendedCost \
    --group-by Type=SERVICE \
    --output json"
```

Add this alias to your `~/.bashrc` or `~/.zshrc`, then run `source ~/.bashrc` (or equivalent shell).

Step 2: Run the Alias & Save Output

```
aws-cost > cost-report.json
```

Step 3: Python Script to Convert JSON → Filtered Slack Message

Create a script called `aws_cost_slack.py`:

```
import json
import requests

# Load cost report
with open('cost-report.json') as f:
    data = json.load(f)

# Extract relevant data
results = data['ResultsByTime'][0]['Groups']
date_range = data['ResultsByTime'][0]['TimePeriod']
start = date_range['Start']
end = date_range['End']

non_zero_services = []
for group in results:
    service = group['Keys'][0]
    amount = float(group['Metrics']['UnblendedCost']['Amount'])
    if amount > 0:
        non_zero_services.append((service, amount))

# Format for Slack
if not non_zero_services:
    message = f"*AWS Cost Report for {start} to {end}*\nNo charges yesterday. 🎉"
else:
    message = f"*AWS Cost Report for {start} to {end}*"
    for service, amount in sorted(non_zero_services,
key=lambda x: x[1], reverse=True):
        message += f"\n• `{service}`: *${amount:.2f}*"

# Send to Slack
SLACK_WEBHOOK_URL = "https://hooks.slack.com/services/your/webhook/url"

payload = {"text": message}

response = requests.post(SLACK_WEBHOOK_URL, json=payload)
print("Slack Response:", response.status_code, response.text)
```

Step 4: Add to Cron (Optional Automation)

Run daily at 8 AM:

```
crontab -e
```

Add this line:

```
0 8 * * * /bin/bash -c 'source ~/.bashrc && aws-cost >
~/cost-report.json && python3 ~/aws_cost_slack.py'
```

Optional: CSV Export (if needed)

You can also convert `cost-report.json` to CSV using Python:

```
import csv

with open('cost-report.json') as f:
    data = json.load(f)

results = data['ResultsByTime'][0]['Groups']

with open('cost-report.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(['Service', 'Amount'])
    for group in results:
        service = group['Keys'][0]
        amount =
float(group['Metrics']['UnblendedCost']['Amount'])
        if amount > 0:
            writer.writerow([service, f"${amount:.2f}"])
```

Search AWS Cost

21 days left in trial

cloudcost

Messages Add canvas Files +

AWS Cost Report: 57295486402 Wednesday, 7 May

Last Month: \$15.30
This Month: \$9.24
Last 7 days: \$11.14
Yesterday: \$1.82

	Service	Last Month	This Month	Last 7 Days	Yesterday
10	Amazon Elastic Compute Cloud - Compute	\$8.18	\$4.36	\$6.09	\$1.06
11	Amazon EC2 Container Registry (ECR)	\$0.00	\$0.00	\$0.00	\$0.00
12	Amazon Simple Notification Service	\$0.00	\$0.00	\$0.00	\$0.00
13	Amazon Simple Storage Service	\$0.00	\$0.00	\$0.00	\$0.00
14	Amazon Virtual Private Cloud	\$0.00	\$0.00	\$0.00	\$0.00
15	Amazon Simple Queue Service	\$0.00	\$0.00	\$0.00	\$0.00
16	AWS Key Management Service	\$0.00	\$0.00	\$0.00	\$0.00
17	AWS Step Functions	\$0.00	\$0.00	\$0.00	\$0.00
18	AWS Cost Explorer	\$0.05	\$0.16	\$0.21	\$0.00
19	AmazonCloudWatch	\$0.00	\$0.00	\$0.00	\$0.00
20	EC2 - Other	\$4.50	\$3.11	\$3.23	\$0.76
21	AWS Lambda	\$0.00	\$0.00	\$0.00	\$0.00

Message @cloudcost

Slack needs your permission to enable notifications. Enable notifications

AWS Cost Report: 572954864028

Last Month: \$15.30
This Month: \$9.24
Last 7 days: \$11.14
Yesterday: \$1.82

	Service	Last Month	This Month	Last 7 Days	Yesterday
Amazon Elastic Compute Cloud - Compute		\$8.18	\$4.36	\$6.09	\$1.06
Amazon EC2 Container Registry (ECR)		\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Notification Service		\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Storage Service		\$0.00	\$0.00	\$0.00	\$0.00
Amazon Virtual Private Cloud		\$0.00	\$0.00	\$0.00	\$0.00
Amazon Simple Queue Service		\$0.00	\$0.00	\$0.00	\$0.00
AWS Key Management Service		\$0.00	\$0.00	\$0.00	\$0.00
AWS Step Functions		\$0.00	\$0.00	\$0.00	\$0.00
AWS Cost Explorer		\$0.05	\$0.16	\$0.21	\$0.00
AmazonCloudWatch		\$0.00	\$0.00	\$0.00	\$0.00
EC2 - Other		\$4.50	\$3.11	\$3.23	\$0.76
AWS Lambda		\$0.00	\$0.00	\$0.00	\$0.00
Tax		\$2.57	\$1.61	\$1.61	\$0.00

